



Project Report

Solving the 8-Puzzle Problem Using Search Algorithms



Team Members

Donya Sameh Fathy Abdelshafy

Ahmed Ashraf Elsayed Kamal

Ahmed Awad Hassan Mohamed

Omar Hany Fathy Ali

DECEMBER 20, 2025

ENG. OMAR KHALED

Faculty of Computers and Information

1. Introduction

This project focuses on solving the classic 8-Puzzle problem using different search algorithms. The main objective is to compare the performance of uninformed and informed search strategies based on time complexity, space complexity, solution optimality, and path cost.

2. Problem Description (8-Puzzle)

The 8-Puzzle problem consists of a **3×3 grid** containing eight numbered tiles (1–8) and one empty space (represented as 0). The tiles can be moved by sliding a tile into the empty position.

Initial State

The puzzle starts from a given initial configuration where the tiles are arranged in a random order.

Goal State

The objective is to reach a predefined goal configuration where the tiles are arranged in ascending order and the empty space is in the correct position.

Example goal state:

1 2 3

4 5 6

7 8 0

Allowed Moves

The empty tile can move:

- Up
- Down
- Left
- Right

Moves are only allowed if they remain within the grid boundaries.

3. State Representation

Each puzzle state is represented as a **2D list (3×3 matrix)** or an equivalent immutable structure (such as a tuple) to allow hashing.

Each state includes:

- The board configuration

- A reference to the parent state
- The move that led to the current state
- Depth of the state
- Path cost

This representation was chosen because:

- It accurately models the puzzle structure
- It allows fast comparison between states
- It supports efficient storage in visited sets and priority queues

Using immutable representations ensures that states can be uniquely identified and avoids revisiting the same configuration multiple times.

4. Implemented Algorithms Overview

- BFS
- DFS
- UCS
- IDS
- A*
- Hill Climbing

5. Algorithm Analysis

5.1 Breadth-First Search (BFS)

5.1.1 Algorithm Description

- **The State-Space Graph**

*In this algorithm, the 8-puzzle is treated as a **graph**:*

Nodes: Each node represents a unique arrangement of the numbers 1–8 and the empty space.

Edges: An edge exists between two nodes if you can reach one from the other by moving the empty tile (Up, Down, Left, or Right).

- **Level-by-Level Exploration**

BFS is "exhaustive." It starts at the initial configuration (Level 0) and generates all possible moves from there (Level 1). Then, it generates all possible moves from every state in Level 1 to create Level 2, and so on.

5.1.2 Time Complexity

The time complexity of BFS is generally expressed in terms of the branching factor and the depth of the solution.

2.1 Branching Factor (p)

The branching factor represents the average number of successor nodes generated from a single state. In an 8-puzzle:

- **Corner positions:** 2 possible moves.
- **Edge positions:** 3 possible moves.
- **Center position:** 4 possible moves.
- **Average:** The effective branching factor b is approximately 3.

2.2 Depth (d)

The depth represents the number of levels (moves) from the initial state to the goal state. Because BFS explores level-by-level, it must generate all nodes at depth $d-1$ before reaching the first goal at depth d .

2.3 Mathematical Derivation

The total number of nodes generated in the worst case is the sum of a geometric series:

$$T(n) = 1 + b + b^2 + b^3 + \dots + b^d$$

Using the geometric series formula:

$$T(n) = \{b^{d+1} - 1\} / \{b - 1\}$$

In Big O notation, we drop lower-order terms and constants, resulting in: **Time Complexity** = $O(b^d)$

5.1.3 Space Complexity

In BFS, time and space complexity are identical: $O(b^d)$. Every node generated during the time-intensive search must be stored in the Frontier (Queue) or the Explored Set (Hash Map) to ensure optimality and prevent infinite loops. This makes memory the primary bottleneck for BFS in larger puzzles (like the 15-puzzle).

Note: Considered High

5.1.4 Solution Optimality

Because it checks every possible move at depth n before looking at depth $n+1$, the first time it hits the goal, it is guaranteed to be the shortest path (the minimum number of moves).

5.1.5 Path Cost

In BFS, the **path cost** represents the total number of moves taken to transition from the initial configuration to the goal state.

5.2 Depth-First Search (DFS)

5.2.1 Algorithm Description

DFS is an uninformed search algorithm that explores the puzzle's state space by going as deep as possible along each branch before backtracking. It uses a Stack (LIFO) data structure to manage the frontier. In this implementation, a "visited set" is used to store the hash of board configurations to prevent the algorithm from getting stuck in infinite loops or cycles.

5.2.2 Time Complexity

- **Theoretical:** $O(b^m)$, where b is the branching factor and m is the maximum depth of the search tree.
- **Practical:** In the 8-puzzle, DFS can be very slow if it chooses a deep path that doesn't lead to a solution quickly, as it may explore many unnecessary states before backtracking.

5.2.3 Space Complexity

- **Low:** The space complexity is $O(bm)$. Unlike BFS, DFS only needs to store the path from the root to the current node, plus the unexpanded siblings of each node on the path.

5.X.4 Solution Optimality

- **No:** DFS does not guarantee an optimal (shortest) solution. It returns the first path it finds to the goal, which might be much longer than the minimum number of moves required.

5.X.5 Path Cost

- **High:** Because it does not guarantee optimality, the path cost (number of moves) is typically very high compared to BFS or A*.
-

5.3 Uniform-Cost Search (UCS)

5.3.1 Algorithm Description

The Step-by-Step Process:

Step 1: Initialization

- Create a node for the Initial State.
- Set its cost $g(\text{initial}) = 0$.

- Insert this node into the Priority Queue.
- Initialize the Explored Set as empty.

Step 2: Selection

- If the Priority Queue is empty, the puzzle is unsolvable.
- Otherwise, Pop the node n with the lowest $g(n)$ from the Priority Queue.

Step 3: Goal Test

- Check if state n matches the Goal State.
- If yes: Success! Trace the parent pointers back to the root to reconstruct the move sequence.
- If no: Proceed to expansion.

Step 4: Expansion

- Identify the position of the blank tile (0).
- Generate all valid moves (Up, Down, Left, Right).
- For each valid move, create a Child State n' .
- Calculate $g(n') = g(n) + 1$.

Step 5: Filtering & Update

For each Child State n' :

- If n' is not in the Explored Set and not in the Priority Queue:
- Add n' to the Priority Queue.
- If n' is already in the Priority Queue with a higher cost:
- Replace that entry with the new, lower-cost path (though in 8-puzzle, since all edges are 1, the first time you see a state in a BFS/UCS manner, it is usually via the shortest path).

Step 6: Finalize Node

- Add the current state n to the Explored Set.
- Return to Step 2.

5.3.2 Time Complexity

Theoretical Complexity

The theoretical time complexity of UCS is defined by the branching factor b and the cost of the optimal solution C^* . Let ϵ be the minimum cost of any edge (in 8-puzzle, $\epsilon = 1$).

- **Time Complexity:** $O(b^{1 + C^*/\epsilon})$. In the 8-puzzle, b is roughly 3 (the average number of available moves), and C^* is the number of moves to the goal. This simplifies to $O(b^d)$, where d is the depth of the solution.
- **Space Complexity:** $O(b^d)$, as all generated nodes in the frontier and explored set must be stored in memory.

Practical Complexity

In practice, the 8-puzzle has a total of $9! / 2 = 181,440$ reachable states.

1. **Uninformed Exploration:** Because UCS is uninformed, it explores states in "concentric circles" of increasing cost. It does not prioritize moves that look closer to the goal.
2. **Worst Case:** In the worst-case scenario (a goal state at maximum depth), UCS may visit nearly all 181,440 states before finding the solution.
3. **Search Overhead:** Each step involves a $O(\log N)$ priority queue operation and a $O(1)$ hash set lookup. While N is relatively small for an 8-puzzle, this becomes a significant bottleneck for larger puzzles (like the 15-puzzle), where the state space grows to over 10^{13} states

5.3.3 Space Complexity

The space complexity of UCS is considered High $O(b^d)$. Unlike algorithms that can "forget" branches (like Depth-First Search), UCS must keep every generated node in memory.

- **The Frontier:** Every state discovered but not yet expanded remains in the priority queue.
- **The Explored Set:** To guarantee optimality and prevent infinite loops, every single state ever visited must be stored.
- **Memory Exhaustion:** For an 8-puzzle, we can store the 181,440 states in modern RAM easily. However, for a 15-puzzle, the number of states exceeds 10^{13} , which would require petabytes of memory, making UCS practically impossible to use without heuristics.

5.3.4 Solution Optimality

If the algorithm reaches the goal state, you can be certain that no other sequence of moves exists that is shorter than the one found. This is because if a shorter path existed, UCS would have found and explored it earlier, as its $g(n)$ would have been smaller. This is what makes UCS **optimal**.

5.3.5 Path Cost

1. Cumulative Nature of the Cost

The path cost, denoted as $g(n)$, represents the **total weight** from the start node to the current node n . Unlike some search algorithms that only look at the "next step," UCS considers the entire history of the path.

2. Uniform Edge Weights in 8-Puzzle

In the 8-puzzle specifically, every single action (sliding a tile) is considered to have a cost of **1**. Because all edges have the same weight, the path cost $g(n)$ is mathematically equivalent to the **depth** of the node in the search tree.

$g(n)$ = number of moves from start to state n

3. The "Cheapest-First" Strategy

The algorithm uses the path cost to prioritize exploration:

- It maintains a frontier where states are sorted by their $g(n)$ value.
 - By always picking the state with the **lowest $g(n)$** , UCS ensures that it never explores a deeper (more expensive) path until all shallower (cheaper) paths have been exhausted.
-

5.4 Iterative Deepening Search (IDS)

5.4.1 Algorithm Description

IDS is a strategy that combines the benefits of DFS's low memory usage with BFS's optimality. It repeatedly performs a **Depth-Limited Search (DLS)**, starting with a limit of 0 and incrementing the limit in each iteration (0, 1, 2, ...) until a goal is found. This ensures that the algorithm finds the shallowest goal state first.

5.4.2 Time Complexity

- **Theoretical:** $O(b^d)$, where d is the depth of the shallowest solution.
- **Practical:** Although it re-explores upper levels of the tree multiple times, the overhead is relatively small because the number of nodes grows exponentially with depth; the final level dominates the total time.

5.4.3 Space Complexity

- **Low:** The space complexity is $O(bd)$, like DFS. It only stores a single path at a time, making it much more memory-efficient than BFS.

5.4.4 Solution Optimality

- **Yes:** IDS guarantees an optimal solution for the 8-puzzle because it explores all nodes at depth n before moving to depth $n+1$.

5.4.5 Path Cost

- **Optimal:** The path cost will be the minimum number of moves required to solve the puzzle from the initial state, matching the cost found by BFS
-

5.5 A* Search

5.5.1 Algorithm Description

A* is an informed search algorithm that combines the path cost and a heuristic function to guide the search toward the goal.

It uses the evaluation function:

$$\text{✚ } f(n) = g(n) + h(n)$$

- where $g(n)$ is the cost from the start state and $h(n)$ is a heuristic estimate of the remaining cost.

- The algorithm always expands the state with the lowest $f(n)$ value.

5.5.2 Time Complexity

In the worst case, A* has exponential time complexity.

However, in practice, using heuristics like Manhattan Distance reduces the number of expanded nodes and improves performance.

5.5.3 Space Complexity

The space complexity of A* is High because it stores all generated states in memory.

5.5.4 Solution Optimality

A* guarantees an optimal solution when an admissible heuristic is used.

The heuristics used in this project do not overestimate the cost, so A* always finds the optimal path.

5.5.5 Path Cost

The path cost represents the number of moves required to reach the goal state.

Since each move has a cost of 1, A* returns the minimum number of moves needed to solve the puzzle.

5.6 Hill Climbing

5.6.1 Algorithm Description

- Hill Climbing is a **local search algorithm** that starts from an initial state and iteratively moves to a neighboring state with a better heuristic value. At each step, the algorithm evaluates all possible successor states and selects the one that appears to be closest to the goal according to a heuristic function (such as Manhattan Distance).
- The algorithm continues moving “uphill” toward better states until no neighboring state has a better heuristic value than the current one. At this point, the algorithm terminates.
- Hill Climbing does not maintain a search tree or explore multiple paths simultaneously; instead, it focuses only on the current state, making decisions based on local information.

5.6.2 Time Complexity

Theoretical:

The time complexity depends on:

- The branching factor (number of possible moves)
- The number of steps until a local optimum is reached

In the worst case, Hill Climbing may examine all neighbors at each step:

$$O(b \times d)$$

where:

- ***b*** is the branching factor
- ***d*** is the depth of the local search

Practical:

In practice, Hill Climbing is usually fast, because:

- It explores only one path
- It does not backtrack
- It does not store previously visited states

However, it may terminate early at a local maximum, plateau, or ridge, even if the global optimum has not been reached.

5.6.3 Space Complexity

Hill Climbing has **Low space** complexity.

It only stores:

- The current state
- Its neighboring states (temporarily)

$$O(b)$$

No search tree, queue, or stack is maintained, making it memory-efficient compared to other search algorithms.

5.6.4 Solution Optimality

Hill Climbing **does not guarantee an optimal solution**.

Because it relies solely on local information, the algorithm can get stuck in:

- Local maxima

- Plateaus
- Ridges

As a result, it may return a suboptimal solution or fail to reach the goal state entirely.

5.6.5 Path Cost

The path cost in Hill Climbing is **not guaranteed to be minimal**.

Since the algorithm does not consider the full path history or compare alternative paths, the number of moves taken to reach a solution may be higher than the optimal path cost found by algorithms such as BFS or A*.

The reported path cost represents only the cost of the locally chosen path, not the globally optimal one.

6. Experimental Results

In this section, all implemented algorithms were executed on the same initial puzzle configuration. For each algorithm, the following metrics were recorded:

- Execution time
- Number of expanded nodes
- Path cost
- Solution depth
- Optimality

The results were organized into comparison tables and visual charts to clearly show performance differences between algorithms.

Initial Puzzle Configuration

1 | 2 | 3

4 | 0 | 6

7 | 5 | 8

6.1 Overall Comparison Table

Algorithm	Status	Expanded Nodes	Execution Time (ms)	Path Cost	Optimal
BFS	Solved	9	0.12	2	Yes
DFS	Solved	329	3.29	320	No
UCS	Solved	9	0.15	2	Yes
IDS	Solved	4	0.06	2	Yes
A* (Manhattan)	Solved	3	0.10	2	Yes
A* (Misplaced)	Solved	3	0.06	2	Yes
Hill Climbing	Solved	3	0.09	2	No

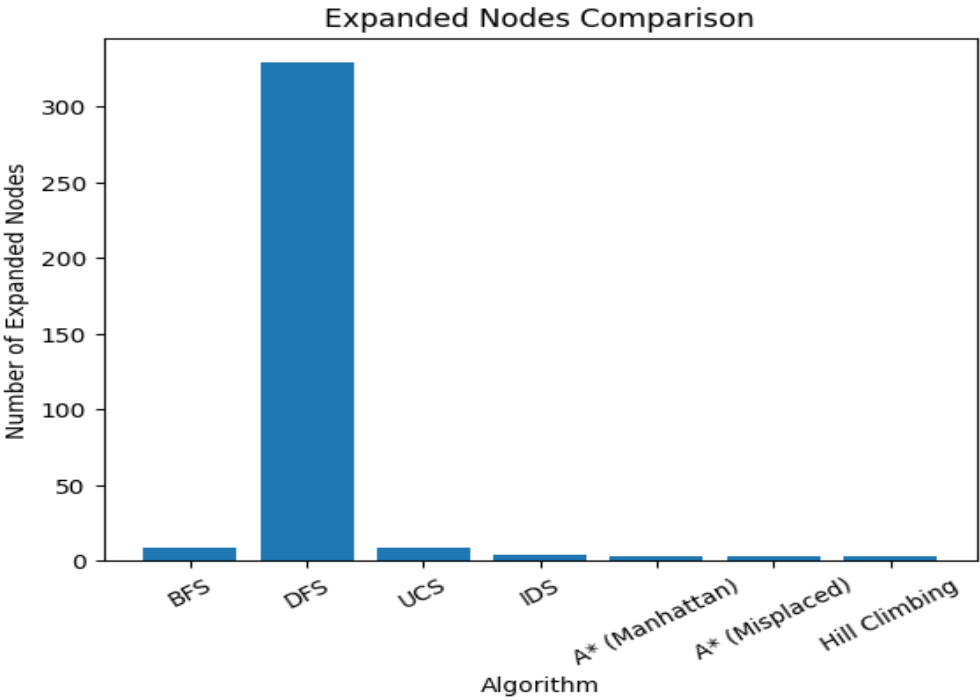
Note:

- **Expanded Nodes:** Total number of explored states
- **Execution Time:** Measured in milliseconds
- **Path Cost:** Number of moves
- **Optimal:** Whether the algorithm guarantees shortest path

6.2 Expanded Nodes Analysis

Algorithm	Expanded Nodes
BFS	9
DFS	329
UCS	9
IDS	4
A* (Manhattan)	3
A* (Misplaced)	3
Hill Climbing	3

Figure 6.1: Expanded Nodes Comparison



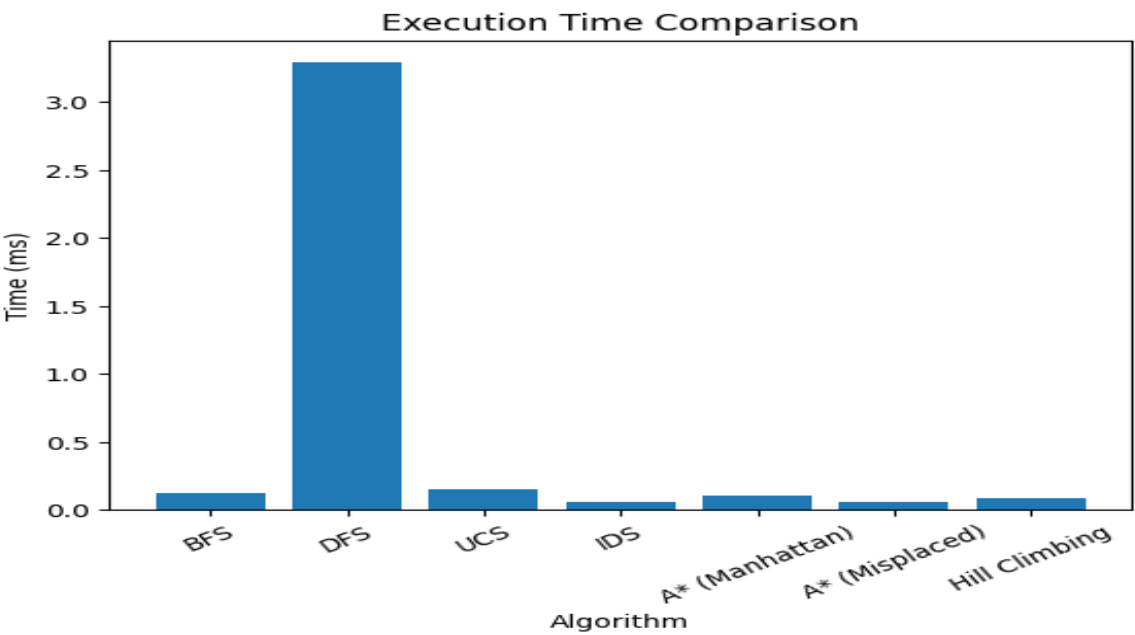
Interpretation:

The results clearly show that heuristic-based algorithms such as A* significantly reduce the number of expanded nodes. DFS performs poorly, expanding a very large number of nodes due to its uninformed deep exploration strategy.

6.3 Execution Time Analysis

Algorithm	Time (ms)
BFS	0.12
DFS	3.29
UCS	0.15
IDS	0.06
A* (Manhattan)	0.10
A* (Misplaced)	0.06
Hill Climbing	0.09

Figure 6.2: Execution Time Comparison



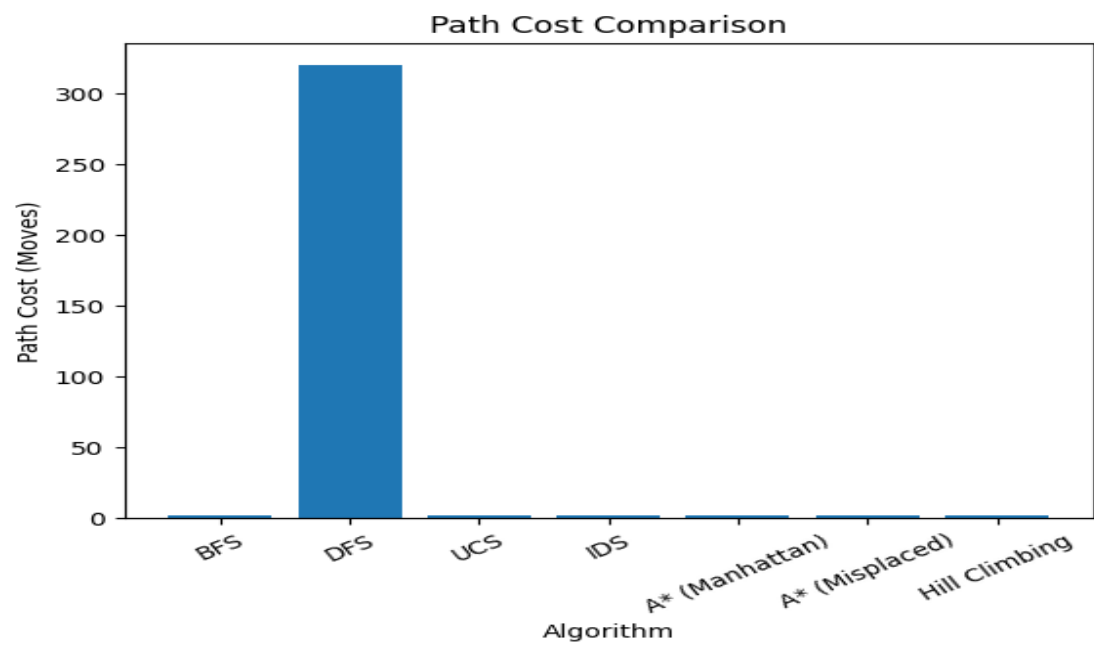
Interpretation:

DFS has the highest execution time due to deep and unnecessary exploration. IDS and A* with Misplaced Tiles heuristic achieve the fastest performance for this puzzle configuration

6.4 Path Cost Analysis

Algorithm	Path Cost
BFS	2
DFS	320
UCS	2
IDS	2
A* (Manhattan)	2
A* (Misplaced)	2
Hill Climbing	2

Figure 6.3: Path Cost Comparison



Interpretation:

BFS, UCS, IDS, and A* always find the optimal path cost. DFS produces a significantly higher path cost, while Hill Climbing may find an optimal solution in simple cases but does not guarantee optimality in general.

7. Discussion

The experimental results confirm that informed search algorithms outperform uninformed ones in terms of efficiency. A* achieved the best overall performance by expanding the fewest number of nodes while maintaining optimality. DFS performed the worst due to its deep exploration behavior, leading to excessive node expansion and high path cost. IDS provided a balanced solution by combining optimality with low memory usage. Hill Climbing was fast and efficient but remains unreliable due to its lack of global optimality guarantees.

8. Conclusion

This project demonstrated how different search algorithms behave when solving the same problem. The results confirm that:

- No single algorithm is optimal in all situations.
- Informed search algorithms, especially A*, provide the best trade-off between performance and optimality.
- Memory usage is a critical factor in search-based problem solving.

The 8-Puzzle problem proved to be an effective benchmark for understanding and comparing AI search strategies.

9. Team Contributions

- Donya Sameh Fathy Abdelshafy: **A*** Implementation and Analysis
- Ahmed Ashraf Elsayed Kamal: **Hill Climbing** Implementation and Analysis
- Ahmed Awad Hassan Mohamed: **DFS & IDS** Implementation and Analysis
- Omar Hany Fathy Ali: **BFS & UCS** Implementation and Analysis